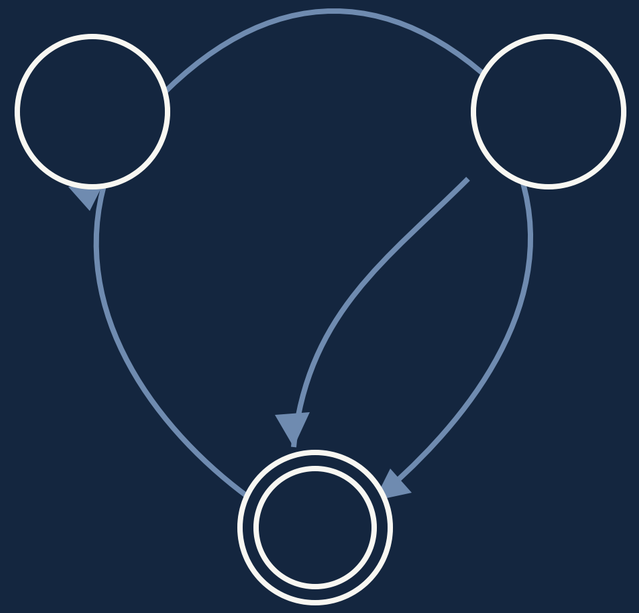
Procesos Estocásticos · Trabajo Final

Simulador de cadenas de Markov

Distribución estacionaria y mixing time



Juan José Ospina y Geronimo Gaviria · 11 de noviembre de 2026

Qué hace el programa

1

Recibe

La matriz estocástica, y la valida

2

Decide

Si se alcanza un estado estable

3

Calcula π

La distribución de estado estable

4

Mide

El mixing time para cada ε

5

Simula

Compara el azar con la teoría

Si el paso 2 da negativo, el programa explica la causa y se detiene.

Cómo está construido

Interfaz

simulador/app.py

Streamlit y Altair: tabla editable, gráficas, grafo y animación

Matemática

simulador/markov.py

NumPy: clases, período, π, mixing time y simulación

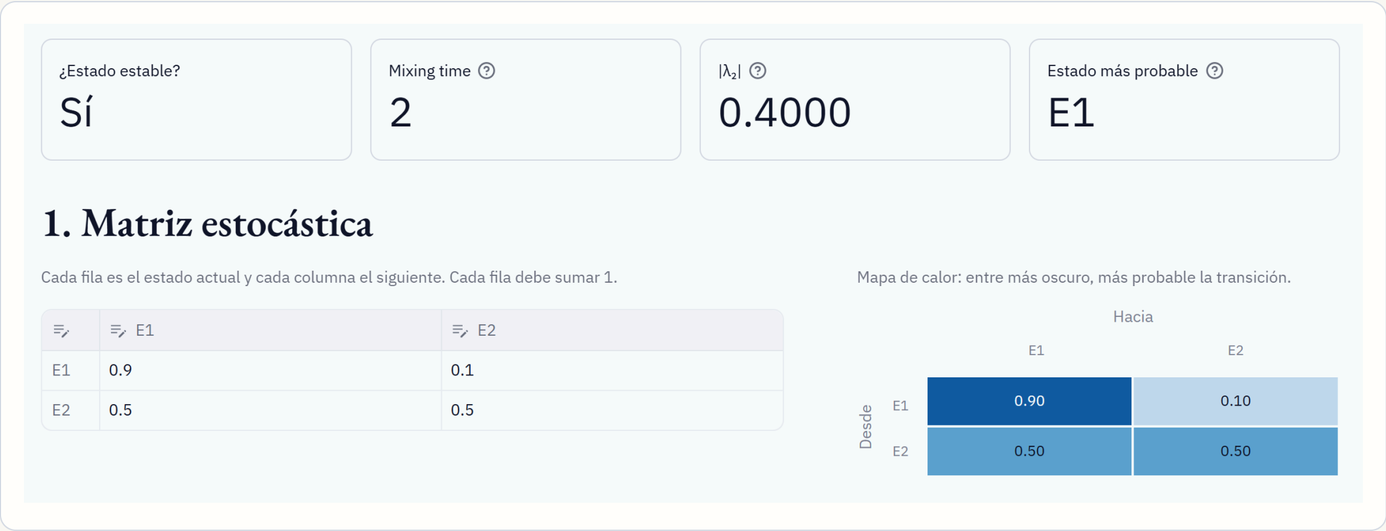
Pruebas

pruebas/

pytest: 48 pruebas automáticas sobre casos conocidos

La matemática no depende de la interfaz, así que se puede probar por separado.

Recorrido por la página



Barra lateral
Ejemplo y tolerancia ε

Resumen arriba
Las respuestas en cuatro tarjetas

Cinco secciones
Una por ítem, con su «¿Qué significa?»

La página se lee de arriba abajo, en el mismo orden de los cinco ítems del enunciado.

Dónde está cada ítem en el código

Ítem	Funciones en markov.py	Qué resuelven
1	leer_matriz, validar_matriz	Leen y validan la entrada
2	analizar_cadena, clases_comunicantes, _periodo	Deciden si hay estado estable
3	distribucion_estacionaria	Calcula π
4	mixing_time, distancia_tv, segundo_valor_propio	Calculan d(t), el mixing time y $\lvert\lambda_2\rvert$
5	simular, evolucion_exacta	Comparan simulación y teoría

Cada función hace una sola cosa y tiene sus propias pruebas.

Conceptos que usa el programa

Matriz estocástica
P(i, j) es la probabilidad de pasar de i a j; cada fila suma 1

Clase comunicante
Grupo de estados que pueden alcanzarse entre sí

Clase cerrada
Clase de la que la cadena ya no puede salir

Estado transitorio
Su probabilidad tiende a 0, porque la cadena termina abandonándolo

Período
MCD de las longitudes de los caminos que regresan a un estado

Distribución estacionaria
Distribución π que no cambia al aplicar P

Ítem 1: entrada de la matriz

	E1	E2	E3
E1	1/3	1/3	1/3
E2	1/3	1/3	1/3
E3	1/3	1/3	1/3

Acepta

0.25, 0,25 y fracciones como 1/3

Valida

Sin vacíos ni negativos, y cada fila suma 1

Señala

La fila o la casilla exacta que falla

Ítem 2: cuándo hay estado estable

Exactamente una clase cerrada, y que sea aperiódica.

Caso	¿Se alcanza?	Por qué
Una clase cerrada, período 1	Sí	Olvida el inicio
Una clase cerrada, período 2 o más	No	La cadena oscila
Dos o más clases cerradas	No	Depende del inicio

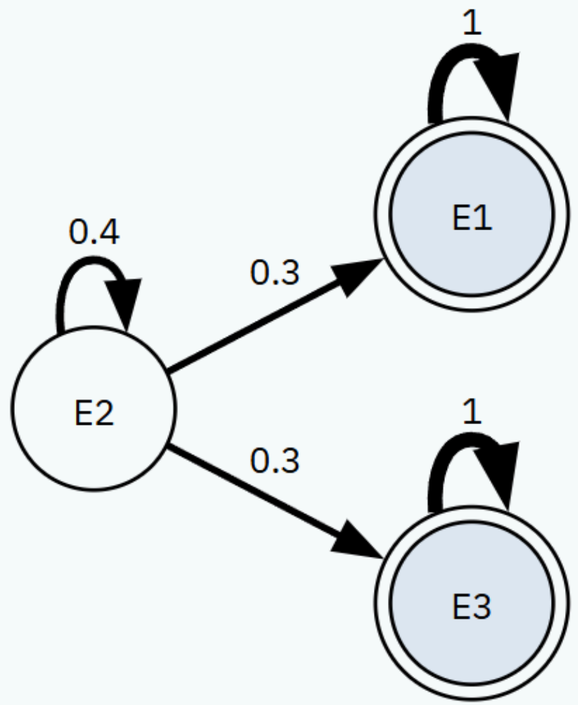
Ítem 2: cómo lo decide el programa

1. Construye el grafo
Hay flecha i → j cuando P(i, j) > 0

2. Agrupa en clases
Estados que se alcanzan mutuamente (Warshall)

3. Busca las clases cerradas
Aquellas de las que no sale ninguna flecha

4. Calcula el período
MCD de las longitudes de los ciclos (BFS)



Dos clases cerradas: no se alcanza

Ítem 2: el período con dos ejemplos

Cadena que oscila

`[[0, 1], [1, 0]]`

Ciclo E1 → E2 → E1, de longitud 2

$$MCD(2) = 2$$

Período 2: desde E1 pasa por (1, 0), (0, 1), (1, 0)… sin estabilizarse

Cadena aperiódica

`[[0, 1, 0], [½, 0, ½], [1, 0, 0]]`

Ciclos E1 → E2 → E1 (2) y E1 → E2 → E3 → E1 (3)

$$MCD(2, 3) = 1$$

Período 1: converge, aunque ningún estado tenga autolazo

Un autolazo basta para ser aperiódica, pero no es necesario.

Ítem 3: distribución de estado estable

$$\pi P = \pi,\ \sum \pi = 1$$

Cómo se resuelve

Sistema lineal con mínimos cuadrados, porque hay más ecuaciones que incógnitas

Ejemplo

P = [[0.9, 0.1], [0.5, 0.5]] da π = (5/6, 1/6), es decir (0.8333, 0.1667)

Ítem 4: cómo calculamos el mixing time

Objetivo: medir cuántos pasos necesita la cadena para olvidar su estado inicial y quedar lo bastante cerca de π.

Distancia entre dos distribuciones

$$\|\mu - \nu\|_{TV} = \tfrac{1}{2} \sum |\mu(x) - \nu(x)|$$

Peor caso entre todos los estados iniciales

$$d(t) = \max_x \|P^t(x, \cdot) - \pi\|_{TV}$$

Primer paso que baja de la tolerancia

$$t_{mix}(\varepsilon) = \min \{ t \geq 0 : d(t) \leq \varepsilon \}$$

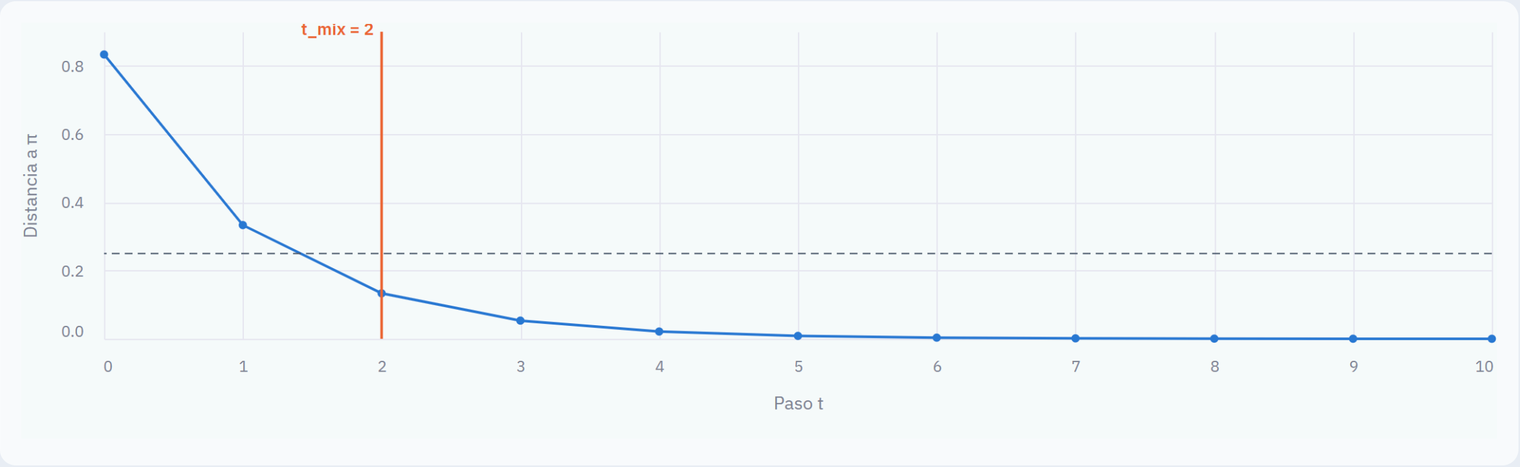
Se itera desde la matriz identidad, con ε = 1/4 por defecto y un límite de 100 000 pasos.

Ítem 4: el algoritmo en código

```
potencia = identidad(n)
para t desde 0 hasta max_pasos:
    d = max(distancia_tv(potencia, pi, por filas))
    si d ≤ ε: devolver t
    potencia = potencia @ P
devolver "no se alcanzó"
```

Cada fila de **potencia** es la distribución después de t pasos si la cadena arrancó en ese estado.

Ítem 4: ejemplo resuelto a mano



t	d(t)	¿≤ 1/4?
0	0.8333	No
1	0.3333	No
2	0.1333	Sí

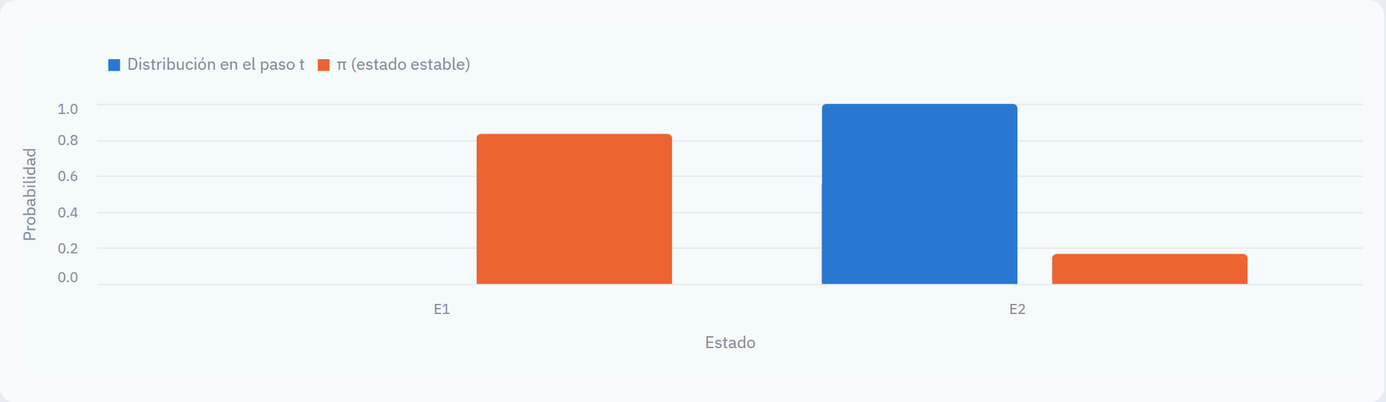
Con P = [[0.9, 0.1], [0.5, 0.5]], el programa da **mixing time = 2**, igual que el cálculo a mano.

Ítem 4: el mismo ejemplo, paso a paso

t	Inicio	Distribución	Distancia a π
1	E1	(0.90, 0.10)	0.0667
1	E2	(0.50, 0.50)	0.3333, peor caso
2	E1	(0.86, 0.14)	0.0267
2	E2	(0.70, 0.30)	0.1333, peor caso

Fila E2 en t = 1: $\tfrac{1}{2}\left(|0.50 - 0.8333| + |0.50 - 0.1667|\right) = 0.3333$

Ítem 4: la convergencia en movimiento



t	Desde E2	Distancia
0	(0, 1)	0.8333
1	(0.5, 0.5)	0.3333
2	(0.7, 0.3)	0.1333

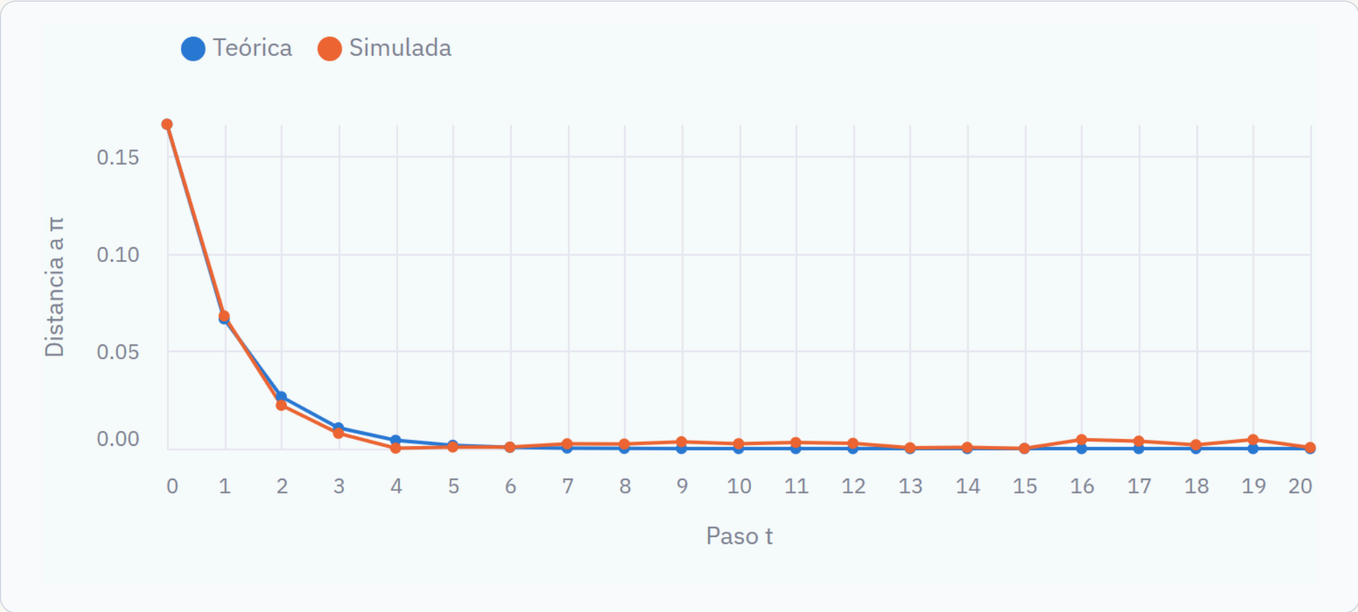
Azul: la distribución en el paso t. Naranja: π. Con ▶ Reproducir, la azul alcanza a la naranja.

Ítem 4: qué explica la velocidad

Entre más cerca de 1 está el segundo valor propio, más lenta es la mezcla.

Cadena de ejemplo	$\lvert\lambda_2\rvert$	Mixing time
Regular de 3 estados	0.3	1
Regular de 2 estados	0.4	2
Absorbente con transitorios	0.5	5

Ítem 5: simulación Monte Carlo

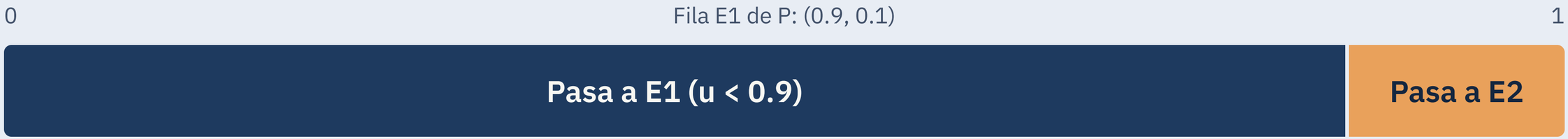


Estado	π	Simulada
E1	0.8333	0.8339
E2	0.1667	0.1661

Cadena regular de 2 estados con 10 000 caminantes y 20 pasos: el azar llega a la misma distribución que el álgebra.

Ítem 5: cómo salta cada caminante

Se sortea un número u entre 0 y 1, y el tramo donde cae decide el siguiente estado.



u = 0.37: cae en el primer tramo, sigue en E1

u = 0.95: cae en el segundo tramo, va a E2

Ítem 5: el efecto de los caminantes

Caminantes	Distancia a π
100	0.0560
1 000	0.0193
10 000	0.0061
100 000	0.0016

≈ 3 veces

menos ruido cada vez que los caminantes se multiplican por 10

Cadena regular de 3 estados, 20 pasos desde E1. Promedio de los últimos 10 pasos, cuando la cadena ya mezcló.

Cómo validamos el programa

48

Pruebas automáticas

Casos de resultado conocido, de la validación a la simulación

16/19

Errores detectados

Rompimos el código a propósito para comprobar las pruebas

2

Cadenas a mano

Mixing time calculado sin el programa, y coincide

También probamos la interfaz en un navegador real: edición, errores y gráficas.

Alcance y límites del programa

Aspecto	Límite	Por qué
Estados	Hasta 12	La tabla sigue legible
Grafo	Hasta 8 estados	Evita flechas ilegibles
Mixing time	100 000 pasos	Avisa si no alcanza ε
Tolerancia ε	De 0.001 a 0.5	Evita el piso numérico
Simulación	1000 pasos	Mantiene la app ágil
Animación	Hasta 60 pasos	La convergencia ya se ve

Trabaja con cadenas de estados finitos y probabilidades que no cambian con el tiempo.

Demostración en vivo

1. Regular (2 estados)

Resumen, mapa de calor y la animación ▶

2. Periódica (2 estados)

Período 2: explica por qué no converge

3. Dos estados absorbentes

Grafo con dos clases cerradas

4. Personalizada

Una matriz propuesta por el profesor

Conclusiones

Diagnostica	Decide si hay estado estable y explica por qué no cuando falla
Calcula	Entrega π y el mixing time, comprobados con casos a mano
Verifica	Contrasta la teoría con una simulación independiente

¿Preguntas?

Anexo: preguntas sobre el método

¿Por qué esa distancia?

Es la mayor diferencia de probabilidad que dos distribuciones asignan a un mismo evento

¿Por qué ε = 1/4?

Es la convención: cada mixing time adicional reduce d(t) al menos a la mitad

¿Por qué d(t) no sube?

Aplicar P a dos distribuciones nunca las aleja, y π no cambia al aplicar P

¿Periódica o reducible?

La periódica tiene un π único pero oscila; con dos clases cerradas hay varios π posibles

Anexo: preguntas sobre el programa

¿Por qué mínimos cuadrados?
Hay n + 1 ecuaciones y n incógnitas; así no hay que escoger cuál eliminar

¿Por qué iterar potencias?
Se necesita d(t) en cada paso, para hallar el primer t y dibujar la curva

¿Por qué semilla fija?
Hace que la simulación sea repetible y que los resultados se puedan verificar

¿Por qué aceptar fracciones?
Tres veces 0.3333 suma 0.9999; tres veces 1/3 suma exactamente 1